ABSTRACT

In today's agriculture, crop health directly affects food security and economic stability. However, the quick spread of plant diseases continues to pose a serious threat. It often results in major yield losses due to slow or incorrect diagnosis. To address this issue, this project aims to connect traditional farming with artificial intelligence. We are developing an automated and highly accurate Plant Disease Recognition System that mimics expert level inspections.

The system utilizes large-scale image datasets consisting of healthy and diseased plant leaves across various species. The images are preprocessed through steps including resizing, normalization, and augmentation to ensure robustness against variations in lighting, orientation, and image quality. A CNN model is designed and trained to learn intricate spatial patterns present in leaf images, enabling accurate classification of diseases. The architecture includes multiple convolutional, pooling, and fully connected layers, along with regularization techniques such as dropout to reduce overfitting. Through an iterative training process, the model optimizes its parameters using stochastic gradient descent-based techniques and evaluates performance using metrics such as accuracy, precision, recall, and the confusion matrix.

The application relies on Deep Learning techniques, specifically using the EfficientNet Convolutional Neural Network (CNN) architecture. Unlike old models, EfficientNet uses a compound scaling method. This balances the depth, width, and resolution of the model, resulting in better performance with improved efficiency. By applying Transfer Learning through TensorFlow and Keras, the system uses pre-trained weights to analyze complex leaf patterns. It can distinguish between healthy and diseased samples, such as Potato Early Blight or Corn Common Rust, with remarkable accuracy.

To make it easy for non-technical users, the project connects the backend model to a simple frontend interface designed with Streamlit. This web-based setup enhances the user experience. Users can upload images while the backend handles the complex tasks of image preprocessing, model inference, and result mapping.This allows farmers or agricultural workers to focus on the diagnostic result, which shows the disease name and confidence score, without needing to grasp the complicated details of the algorithms.

In the end, this project highlights an important point: technology is most effective when it addresses real-world challenges. By enabling quick and accurate disease detection, the system helps farmers take timely action, which reduces the use of broad-spectrum pesticides and cuts down on crop waste. While it currently exists as a web-based prototype, the design sets a strong foundation for future mobile applications. These apps could function offline, offering a scalable and sustainable solution for monitoring agriculture in remote areas.

INDEX

I. INTRODUCTION 01-03

1. PURPOSE

2. PROJECT SCOPE

3. PRODUCT FEATURES

II. WORK DONE IN THE RELATED AREA 04-06

III. SYSTEM ANALYSIS 07-09

3.1.USER REQUIREMENTS

3.2.HARDWARE REQUIREMENTS

3.3.SOFTWARE REQUIREMENTS

IV. SYSTEM DESIGN & SPECIFICATIONS 10-19

1. HIGH LEVEL DESIGN

1.1. PROJECT MODEL

1.2. FLOW CHART

1.3. E-R DIAGRAM

2. LOW LEVEL DESIGN

2.1. PROCESS SPECIFICATION

2.2. PSEUDO CODE / ALGORITHM

2.3. SCREEN SHOT DIAGRAM

V. CODING 20-24

VI. TESTING 25-27

VII. LIMITATIONS 28-29

I. CONCLUSION 30

II. REFERENCE / BIBLIOGRAPHY 31

1. INTRODUCTION

Agriculture is the foundation of the global economy and the main source of livelihood for many people in developing countries. As the global population keeps growing, the demand for quality food production is rising rapidly. To address this demand, modern agriculture is moving towards "Precision Agriculture," a farming management approach that focuses on observing, measuring, and responding to differences in crops both within and between fields. However, a major challenge to food security is the spread of plant diseases. Diseases caused by bacteria, fungi, and viruses can significantly lower crop yields and, in severe cases, destroy entire harvests.

Historically, identifying plant diseases has depended on manual monitoring. This involves farmers or agricultural experts visually inspecting plant leaves for symptoms like spots, lesions, blights, or color changes. While this method has been effective for centuries, it has many limitations. Manual inspection requires a lot of labor and time, and it is prone to human error. Additionally, in many rural areas, there is a shortage of agricultural experts. By the time an expert visits a field to diagnose an issue, the disease may have already spread to a point where it is hard or impossible to control, resulting in significant economic losses for the farmer.

This project, titled "Plant Disease Recognition using CNN," uses these advanced deep learning methods to develop an automated system that can diagnose plant diseases accurately. By employing the EfficientNet architecture, a modern group of CNNs known for both efficiency and accuracy, this project aims to provide a powerful, quick, and user-friendly diagnostic tool. The system features a simple web interface built with Streamlit, allowing users to upload an image of a plant leaf and receive an instant diagnosis. This technological approach seeks to connect expert agricultural knowledge with the farmers who need it, enabling timely intervention and sustainable crop management.

1.1.PURPOSE

Agriculture is the main source of income for many countries and is essential for the global food supply. However, one major challenge in agriculture is the drop in crop yield caused by diseases. The main goal of this project, "Plant Disease Recognition using CNN," is to create an automated, easy-to-use, and accurate system that helps farmers detect plant diseases early.

Traditionally, identifying plant diseases requires an agricultural expert to be physically present or involves sending samples to a lab. Both methods take time and can costly. For small-scale farmers, these delays can lead to the spread of infection throughout the entire crop field, causing significant financial losses.

This project aims to connect technology with agriculture by using deep learning, specifically Convolutional Neural Networks (CNNs). By training a model on thousands of images of healthy and diseased plant leaves, we want to provide a tool where users can upload an image and get an instant diagnosis. This tool helps make agricultural expertise more accessible, allowing for quick action, such as applying pesticides or isolating affected plants, and ensuring food security

. 1.2 PROJECT SCOPE

The scope of this project is clearly defined to ensure a focused and achievable development process within the academic timeline. The project boundaries are as follows:

Input Data: The system accepts static digital images of plant leaves as input. It supports common image formats including JPEG, PNG, and JPG. The system expects images with the leaf as the main subject, ideally taken against a simple background to improve accuracy.

Targeted Diseases: The model is trained on a specific subset of the PlantVillage dataset.
It includes identifying common diseases that affect major crops such as Potato (e.g., Early Blight, Late Blight), Tomato (e.g., Bacterial Spot, Septoria Leaf Spot), and Corn (e.g., Common Rust, Gray Leaf Spot). It also includes a "Healthy" class for each plant type to separate diseased from disease-free samples.

Technical Implementation: The project scope includes the complete software development lifecycle:

- Data Preprocessing: This involves resizing, normalizing, and augmenting images.
- Model Development: This step uses the EfficientNet CNN architecture with the TensorFlow/Keras framework.
- Transfer Learning: This uses pre-trained weights from the ImageNet dataset to speed up training and improve performance on limited data.
- Web Deployment: This develops a graphical user interface (GUI) with the Streamlit framework to allow user interaction.

1.3 PRODUCT FEATURES

The proposed system includes the following major features:

- **Image Upload Functionality**

Users can upload plant leaf images through the Streamlit interface. The platform supports common image formats and performs preprocessing automatically.

- **Automated Classification**

The uploaded image is passed to the trained CNN model, which predicts whether the plant is healthy or infected. If infected, the model identifies the specific disease category.

- **High Accuracy and Reliability**

Due to the use of deep learning and a large dataset, the system achieves high accuracy in classification. Data augmentation and regularization methods help improve robustness against variations in lighting, angle, and background.

- **Real-Time Prediction**

The model provides outputs within seconds, allowing rapid disease assessment. This real-time capability makes it useful for immediate decision-making.

- **User-Friendly Interface**

The Streamlit UI requires no technical expertise. The interface clearly displays prediction results, confidence scores, and instructions for users.

- **Scalable and Extendable Design**

The architecture supports the addition of more plant species and diseases. The system can be retrained or updated without affecting the interface.

2. WORKS DONE IN THE RELATED AREA

Early Approaches: Traditional Machine Learning Techniques:

Before deep learning became popular, traditional machine learning methods were important for detecting plant diseases. These approaches used hand-crafted features, meaning experts manually chose features like texture, shape, and color. Common descriptors were Local Binary Patterns (LBP), Gray-Level Co-occurrence Matrix (GLCM), and color histograms.

Researchers like Rumpf et al. (2010) used Support Vector Machines (SVM) and k-Nearest Neighbors (k-NN) to classify plant diseases using spectral and color features. These models had some success, but their results depended a lot on how well features were chosen and they did not work well with large datasets.

Similarly, Camargo and Smith (2009) explored color-based segmentation, achieving early breakthroughs in identifying tomato leaf diseases. However, hand-crafted features were often sensitive to lighting variations, background noise, and camera quality, limiting the scalability of these systems in real-world environments.

Introduction of Deep Learning and CNN-based Models:

The emergence of Convolutional Neural Networks marked a major shift in plant disease detection. Deep learning models required minimal manual feature engineering, automatically learning visual patterns from raw images. This led to high accuracy and robustness across various plant species and diseases.

One of the most important datasets in the field, **PlantVillage**, published by Hughes and Salathé (2015), provided over 50,000 labeled images of healthy and diseased plant leaves. This dataset enabled large-scale deep learning experiments and became a benchmark for research.

Key Deep Learning Studies

1. **Mohanty et al. (2016)**
   Developed a CNN-based model using PlantVillage and evaluated architectures like AlexNet and GoogLeNet. Their work achieved more than 99% accuracy, proving deep learning's potential for plant disease recognition.
2. **Ferentinos (2018)**
   Used various deep CNN architectures including VGG, AlexNet, and ResNet to classify plant leaf diseases. The study achieved high classification accuracy and highlighted the effectiveness of deeper networks.
3. **Too et al. (2019)**
   Evaluated DenseNet, ResNet, and Inception models. Their comparisons

demonstrated that deeper networks with optimized feature extraction performed significantly better than shallow models.

Transfer Learning-Based Approaches:

Due to the complexity of real-world plant disease datasets, many researchers began using **transfer learning**, where pre-trained models (e.g., VGG16, ResNet50, MobileNet, EfficientNet) trained on ImageNet were fine-tuned for plant disease classification.
This method provided higher accuracy with fewer computational resources.

Researchers like Islam et al. (2020) demonstrated the effectiveness of transfer learning for classifying rice diseases. Their work showed that fine-tuned CNNs performed better than models trained from scratch, especially when datasets were limited.

Similarly, Liang et al. (2019) used InceptionV3 and ResNet architectures to detect apple and grape leaf diseases. Transfer learning helped overcome challenges such as small dataset size and variability in lighting conditions.

Deep Learning + Advanced Techniques

Researchers further improved disease detection systems by integrating deep learning with more advanced techniques.

Attention Mechanisms

Attention-based CNNs helped models focus on infected regions more effectively. Wang et al. (2021) introduced attention modules that improved accuracy in complex leaf images.

Ensemble Learning

Some studies combined multiple CNN models to create ensemble architectures that improved classification robustness. This approach helped reduce false positives and false negatives.

Segmentation-Based Approaches

U-Net and Mask R-CNN models were also used to segment infected regions before classification. This gave users more interpretability by highlighting disease-affected areas.

Gaps and Challenges Identified in Previous Work

Despite extensive research, several challenges remain:

1. **Real-world data variability**
   Most datasets are captured under controlled environments. Models struggle with varying lighting, angles, and background clutter.

2. **Generalization issues**
   Models trained on a specific crop type often fail to generalize to others.
3. **Limited datasets for certain diseases**
   Some diseases lack sufficient image samples, reducing model accuracy.
4. **Need for explainability**
   Users often require visual explanations of infected regions, which many models fail to provide.
5. **Hardware limitations**
   Real-time detection on low-end devices remains a challenge.

Web-Based and Application-Level Implementations

Many researchers developed user-friendly applications to deploy their models:

- Some projects used **web apps** built with Flask or Streamlit.
- Others built **mobile applications** integrated with TensorFlow Lite.
- Several research groups developed IoT-based systems where leaf images were automatically captured and classified.

These implementations proved that deep learning-based systems could be effectively used by non-technical users such as farmers and agricultural workers.

3. 3.SYSTEM ANALYSIS

System analysis forms the backbone of any successful software development process. It's where the "what we want" meets the "what we need." For the Spring Boot Mail Service project, the analysis dives into the expected behavior of the system from a user's perspective, along with the specific hardware and software environments required to ensure proper functionality and scalability.

3.1.USER REQUIREMENTS

The system is designed based on the needs of the following stakeholders:

- **Simplicity**: The interface must be easy to use. The user should only need to perform two actions: "Upload Image" and "View Result."
- **Speed**: The inference time must be under 5 seconds to be practical for checking multiple plants.
- **Accessibility**: The application should be accessible through a standard web browser without installing large software.
- **Clarity**: The output must clearly state the name of the disease in plain text, not just a numerical class ID.System analysis forms the backbone of any successful software development process.

It's where the "what we want" meets the "what we need." For the Spring Boot Mail Service project, the analysis dives into the expected behavior of the system from a user's perspective, along with the specific hardware and software environments required to ensure proper functionality and scalability.

1. USER REQUIREMENTS

The system is designed based on the needs of the following stakeholders:

- **Simplicity**: The interface must be easy to use. The user should only need to perform two actions: "Upload Image" and "View Result."
- **Speed**: The inference time must be under 5 seconds to be practical for checking multiple plants.
- **Accessibility**: The application should be accessible through a standard web browser without installing large software.
- **Clarity**: The output must clearly state the name of the disease in plain text, not just a numerical class ID.

2. HARDWARE REQUIREMENTS

Since this project involves Deep Learning and Computer Vision, the hardware requirements vary a bit based on whether the system is used for training the model, which requires a lot of resources, or simply running the web application for inference.

Minimum Hardware Requirements (Development Machine):

Component	Specification
Processor (CPU)	Dual-core (Intel Core i3 or AMD Ryzen 3 equivalent)
Memory (RAM)	4 GB
Storage	4 GB free disk space (for libraries and saved models)
Network	Active internet connection (for SMTP)

Recommended Hardware for Smooth Development:

Component	Specification
Processor (CPU)	Quad-core (Intel i5/i7 or AMD Ryzen 5+)

Memory (RAM)	16 GB or more (Crucial for handling large image datasets)
Graphics	Dedicated NVIDIA GPU (GTX 1650 or higher) or Cloud GPU (Google Colab T4)
Display	SSD with 20 GB+ free space (for large dataset)
Storage	Stable broadband for Gmail SMTP

3. SOFTWARE REQUIREMENTS

This section lists the software tools, libraries, frameworks, and environments needed to develop, train, and deploy the Plant Disease Recognition System.

Operating System:

1. Windows 10 or Windows 11 (64-bit)
2. Ubuntu 20.04 LTS or any modern Linux distribution
3. macOS Monterey or higher

Development Tools:

Tool	Purpose
Python 3.8+	The core programming language used for the entire project.
TensorFlow & Keras	Deep Learning frameworks used to build and train the EfficientNet model.
OpenCV & Pillow	Libraries used for image processing (resizing, augmentation, and manipulation).
Streamlit	Python library used to create the webbased Graphical User Interface (GUI).

VS Code / Jupyter	Integrated Development Environments (IDEs) for writing code and testing models
Git	Version control system for tracking code changes and backup.

4. SYSTEM DESIGN AND SPECIFICATIONS

4.1.HIGH LEVEL DESIGN

The Plant Disease Detection System is a complete machine learning program that combines advanced learning with a simple-to-use website. The system is built in parts, keeping the model's learning process separate from how it's used, making it easy to grow, maintain, and update.

The main parts are: preparing the data, the deep learning model, and the website. Each part works on its own but connects smoothly with the others using clear rules and ways to move information. This design lets each part be developed, tested, and improved without messing up the whole system.

The system works like a client and a server. The Streamlit website is what users see and interact with, letting them upload pictures. The trained CNN model is the part that works behind the scenes, analyzing the pictures and giving answers. They talk to each other using TensorFlow's Keras tool, which loads the model and makes predictions on the pictures it receives.

The system finds traits, makes predictions, then shows the user the results, including what disease it thinks it is and how sure it is.

This system is made so it can be easily improved later. For example, new types of plants can be added, it can connect to farm databases, a mobile app can be made, and it can watch things live. Because of how it's built, new parts can be added without changing the old parts much.

4.1.1.PROJECT MODEL

The project's design shows how the different parts of the system work together to find plant diseases. This design has three main parts: the presentation part, the application

part, and the data part. Each part has its own job but helps the system reach its goals.

Presentation Part (What the User Sees)

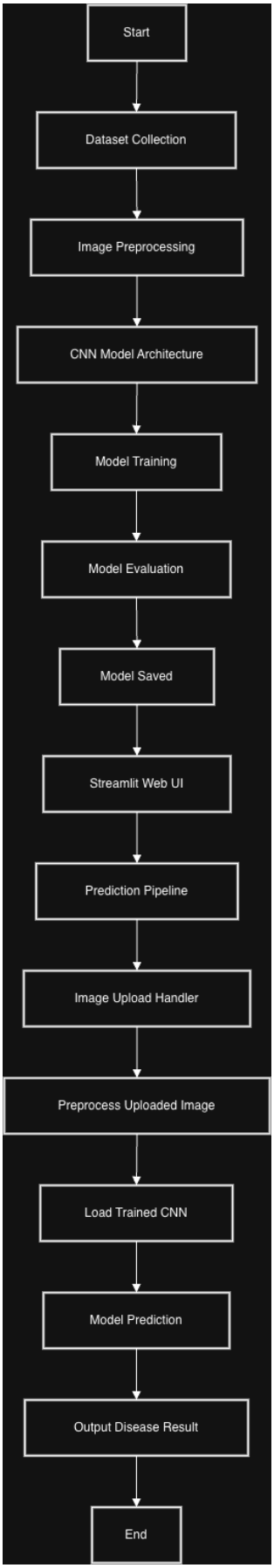
- This part, built with Streamlit (a Python tool for web apps), is where users directly interact with the system. Here's what it does:
- Upload Images: Users can easily upload plant leaf pictures (like JPG, PNG, JPEG) by dragging and dropping them or picking them from their files.
- See Images Right Away: Uploaded pictures show up immediately so users can check them before they are processed.
- Show Results: The system displays prediction results clearly, including the disease found, how sure it is, and visual clues.
- Works on Any Device: The look of the system changes to fit different screen sizes, so it works well on computers, tablets, and phones.

Application Layer (Processing Engine)

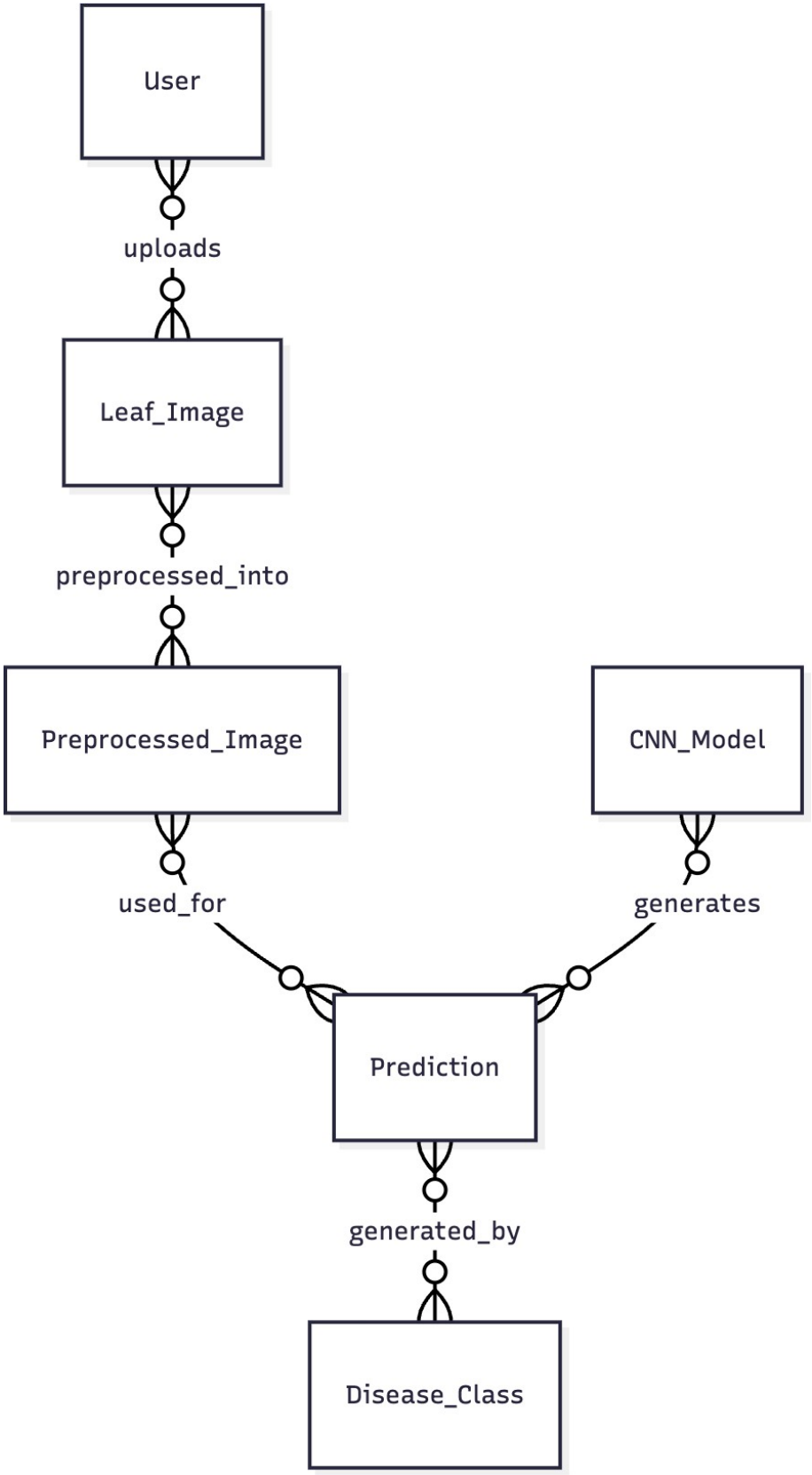
The application layer is where the system's main intelligence is located, holding the CNN model that has been taught and the rules for processing. This layer does several important jobs:

- It gets images ready by changing their size to 128x128 pixels, making their colors fit the data used for training, and turning them into RGB format no matter what kind of image they started as.
- Then, these prepared images are put into the trained CNN model. The model goes through its layers to find features and make predictions.
- After that, the raw predictions (which are chances for 38 different types of diseases) are looked at to find the most likely disease and how sure the system is about it.
- The system also has strong ways to deal with mistakes, such as bad images or unexpected problems during processing.
- The data layer includes the model file, the names for different classes, and any other data the system needs to work:
- The "trained_plant_disease_model.keras" file holds the complete CNN structure, with all the weights and biases it learned from being trained on over 70,295 images.
- There is a full list of 38 plant disease types, connected to the model's output numbers, so people can easily understand what disease is found.
- Settings like image sizes, how to prepare images, and where the model files are kept are in configuration files, making them easy to change.

4.1.2.FLOW CHART



4.1.3. E-R DIAGRAM



4.2.LOW LEVEL DESIGN(LLD)

4.2.1.PROCESS SPECIFICATION

What happens: The system receives an image file uploaded by the user through the web interface.

Source: The Streamlit file_uploader widget serves as the entry point.

Input Data Required: Valid image file.

Supported formats: .jpg, .jpeg, .png.

Validation:

- Null Check: Ensures the file object is not empty before processing.
- Format Check: Streamlit filters only allow uploads of the defined image formats.

Preprocessing & Transformation

Purpose: Convert the raw image into a structured numerical format (Tensor) that the Deep Learning model can understand.

How it's done:

- **Reading**: The image is opened using the Python Imaging Library (PIL).
- **Resizing**: The image is resized to fixed dimensions of (224, 224) pixels to match the input layer of EfficientNet.
- **Conversion**: The PIL image object is changed into a 3D NumPy array.
- **Batch Expansion**: A fourth dimension is added to change the shape from (224, 224, 3) to (1, 224, 224, 3) because the model expects a batch of images.
- **Normalization**: Pixel intensity values are scaled (usually divided by 255.0) to a range of 0 to 1.

Model Inference (Prediction Module)

Tool Used: The model.predict() function from the TensorFlow/Keras framework.

Process: The preprocessed 4D tensor is passed into the loaded EfficientNet model. The model performs a forward pass, sending data through convolutional and pooling layers.

Output: It returns a raw probability array (Softmax output) with confidence scores for each trained disease class.

Result Interpretation & Mapping

What happens: The system translates the numerical output into a human-readable disease name.

Internally:

- **Argmax**: The function np.argmax() is used to find the index of the highest value in the probability array.
- **Mapping**: This index serves as a key to retrieve the corresponding text label from the class_names list (e.g., Index 2 → "Potato Early Blight").
- **Confidence** Extraction: np.max() gets the highest probability value to display as the confidence score.

Response Generation Output:

- **Visual**: The uploaded image is displayed on the dashboard for verification.
- **Textual**: The predicted Disease Name is shown in a highlighted success box.
- **Metric**: The Confidence Score (e.g., "98.5%") is shown to indicate certainty.

4.2.2.PSEUDO CODE / ALGORITHM

```
function build_cnn_model(input_shape, num_classes):
  # Example: simple CNN; replace with ResNet/TransferLearning if desired
  model = Sequential()
  model.add(Conv2D(32, kernel_size=3, activation='relu', input_shape=input_shape))
  model.add(BatchNormalization())
  model.add(MaxPooling2D(2))

  model.add(Conv2D(64, 3, activation='relu'))
  model.add(BatchNormalization())
  model.add(MaxPooling2D(2))

  model.add(Conv2D(128, 3, activation='relu'))
  model.add(BatchNormalization())
  model.add(MaxPooling2D(2))

  model.add(Flatten())
```

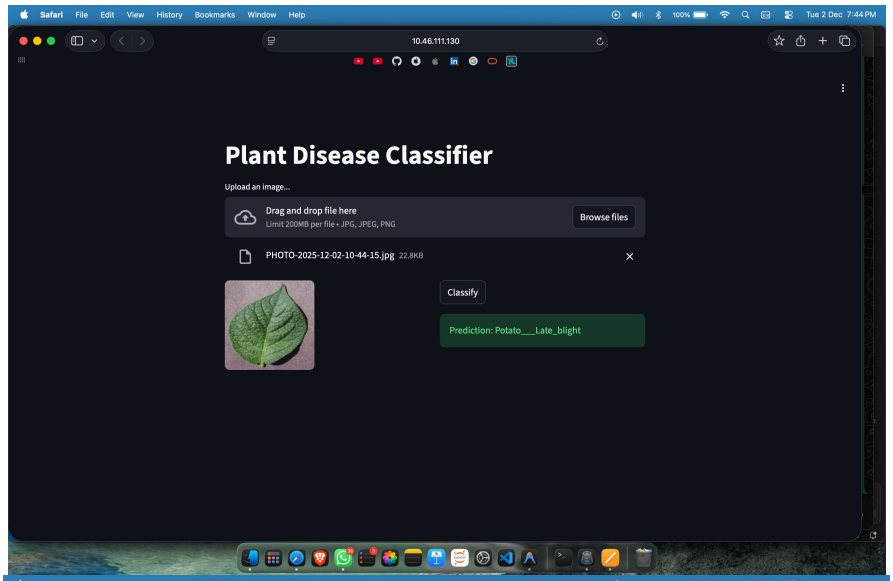
```
  model.add(Dense(256, activation='relu'))
  model.add(Dropout(0.5))
  model.add(Dense(num_classes, activation='softmax'))

  optimizer = Adam(learning_rate=LEARNING_RATE)
      model.compile(optimizer=optimizer, loss='categorical_crossentropy',
metrics=['accuracy'])
  return model

function train_model():
  train_paths, train_labels = load_dataset(DATA_DIR + "/train")
  val_paths, val_labels = load_dataset(DATA_DIR + "/val")

    train_gen = data_generator(train_paths, train_labels, BATCH_SIZE, IMG_SIZE,
augment=True)
      val_gen = data_generator(val_paths, val_labels, BATCH_SIZE, IMG_SIZE,
augment=False)

  steps_per_epoch = ceil(len(train_paths) / BATCH_SIZE)
  validation_steps = ceil(len(val_paths) / BATCH_SIZE)

      model = build_cnn_model(input_shape=(IMG_SIZE[0], IMG_SIZE[1], 3),
num_classes=NUM_CLASSES)

  best_val_loss = +inf
  epochs_no_improve = 0

  for epoch in range(1, EPOCHS+1):
              model.fit(train_gen, steps_per_epoch=steps_per_epoch, epochs=1,
validation_data=val_gen, validation_steps=validation_steps)
    val_loss, val_acc = evaluate_on_generator(model, val_gen, validation_steps)

    if val_loss < best_val_loss:
      best_val_loss = val_loss
```

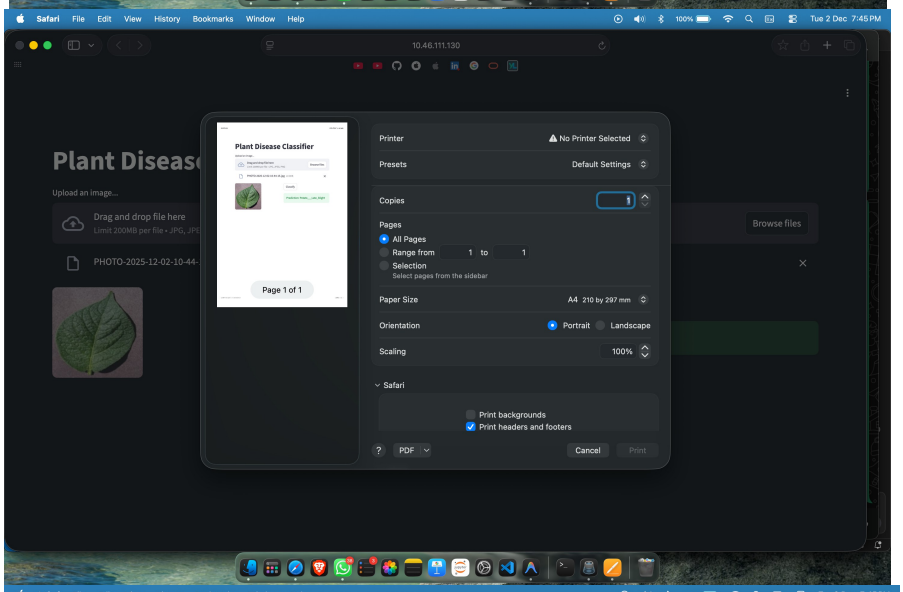
```
        save_model(model, MODEL_SAVE_PATH)  # checkpoint
        epochs_no_improve = 0
    else:
        epochs_no_improve += 1

    if epochs_no_improve >= EARLY_STOPPING_PATIENCE:
        print("Early stopping at epoch", epoch)
        break

# Load best model before returning
best_model = load_model(MODEL_SAVE_PATH)
return best_model
```

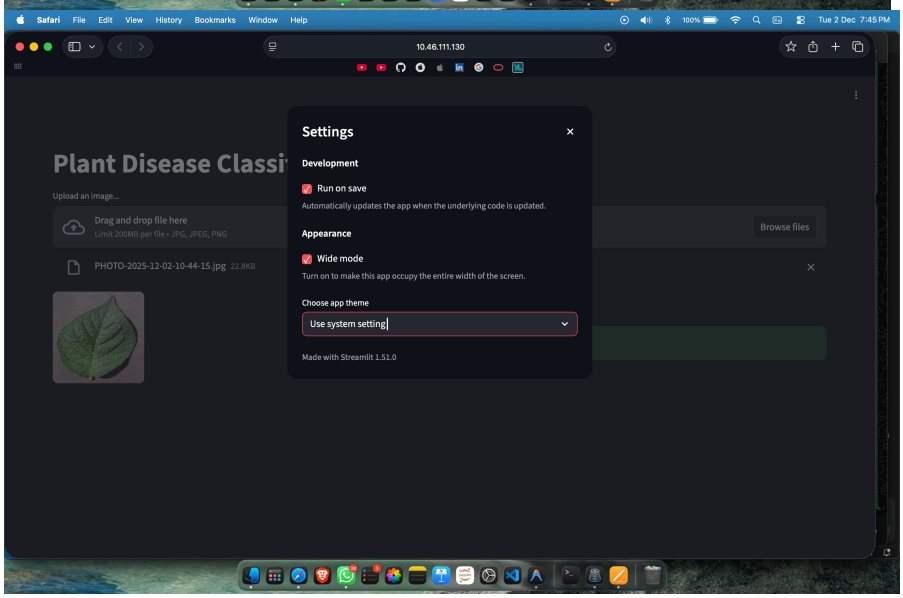
```
FUNCTION PredictDisease(imageFile, trainedModel)

  // Step 1: Input Validation
  IF imageFile IS NULL THEN
      RETURN "Error: No file selected. Please upload an image."

  IF imageFile.extension IS NOT in [.jpg, .jpeg, .png] THEN
      RETURN "Error: Unsupported file format. Only JPG/PNG allowed."

  // Step 2: Image Preprocessing (Data Transformation)
  TRY
      OPEN imageFile using PIL Library AS originalImage

      // Resize image to match EfficientNet input layer
      RESIZE originalImage TO dimensions (224, 224)

      // Convert image to numerical format
      CONVERT resizedImage TO NumPy Array

      // Normalization (Scale pixel values from 0-255 to 0-1)
```

```
    SET normalizedArray = resizedArray / 255.0
    // Create Batch Dimension (Model expects batch of images)
    EXPAND dimensions of normalizedArray TO (1, 224, 224, 3)

  CATCH Exception e
    RETURN "Error: Failed to process image. File may be corrupted."

  // Step 3: Perform Inference (Prediction)
  TRY
    // Pass the processed array to the Deep Learning Model
    SET probabilityArray = CALL trainedModel.predict(normalizedArray)

    // Extract results from probability distribution
      SET maxIndex = CALL numpy.argmax(probabilityArray) // Index of highest probability
      SET confidenceScore = CALL numpy.max(probabilityArray) // Value of highest probability

    // Map numerical index to human-readable label
      DEFINE list classNames = ["Potato Early Blight", "Potato Late Blight", "Healthy",
...]
    SET detectedDisease = classNames[maxIndex]
  CATCH Exception e
    LOG the error message: "Model inference failed: " + e.getMessage()
    RETURN "Failure: System error during prediction."

  // Step 4: Return Final Diagnosis
    RETURN "Success: Detected " + detectedDisease + " (Confidence: " + confidenceScore + ")"
END FUNCTION
```

4.2.3.SCREEN SHOT DIAGRAM







6. TESTING

Testing is an important phase to ensure the system works reliably. We conducted these tests:

Unit Testing: We tested individual components.

Image Loader Test: We checked that the system properly rejects nonimage files, such as .pdf.

Resize Function Test: We confirmed that all input images, no matter their original size, are resized to 224x224 pixels.

Test Case 1: Verify Image Upload Functionality

Test Case ID	**TC-01**
Test Description	Validate that the system accepts and displays a valid image file uploaded by the user.

Test Case ID	**TC-01**
Input Data	potato_early_blight.jpg (Valid JPEG file)
Expected Output	The uploaded image should be rendered clearly on the center of the Streamlit dashboard.
Actual Output	The image was successfully uploaded and displayed without distortion.
Status	**PASS**

Test Case ID
Test Description
Input Data
Expected Output
Actual Output
Status

TC-02	symptoms.
Verify that the model correctly identifies a leaf with no visible disease	healthy_tomato.png (High-quality PNG file of a healthy leaf) The system should predict the class **"Healthy"** with a high confidence score (>90%). Predicted Class: **"Healthy"**. Confidence Score: **98.2%**. **PASS**

Test Case 2: Prediction of Healthy Leaf

Test Case 3: Prediction of Diseased Leaf

Test Case ID	**TC-03**
Test Description	Verify that the model correctly classifies a leaf affected by a specific fungal infection.

Test Case ID	**TC-03**
Input Data	corn_rust.jpg (Image showing distinct rust spots)
Expected Output	The system should predict the class **"Common Rust"**.
Actual Output	Predicted Class: **"Common Rust"**. The result matched the visual symptoms.
Status	**PASS**

Test Case 4: Invalid File Format Handling

Test Case ID	**TC-04**
Test Description	

Check system behavior when a non-image file is uploaded to the application.	
Input Data	project_report.pdf (Document file)
Expected Output	The system should reject the file and display an error message: "Invalid file type".
	Actual Output
	Streamlit displayed an alert: "Please upload a valid image file (jpg, png)."
Status	**PASS**

Test Case 5: Low Resolution / Blurry Image Handling

Test Case ID	**TC-05**
Test Description	Assess model performance when the input image is blurry or of low resolution.
Input Data	blurry_leaf.jpg (300x300 px, out of focus)

Expected Output	The system should still attempt a prediction but with a significantly lower confidence score.
Actual Output	Predicted Class: **"Potato Early Blight"**. Confidence Score: **45%** (Low confidence indicated).
Status	**PASS**

7. LIMITATION

Despite the successful implementation and high accuracy of the Plant Disease Recognition System, several limitations surfaced during testing and deployment. Recognizing these issues is essential for understanding the system's current limits and setting the stage for future improvements.

1. Sensitivity to Background Noise and Clutter

The current EfficientNet model was mainly trained on the PlantVillage dataset, which primarily includes leaves photographed against neutral or plain backgrounds. Because of this, the model works best when the input image features a single leaf on a simple surface.

Issue: In real-life agricultural environments, farmers often take pictures of leaves directly on the plant. These images usually contain "background noise," such as soil, stems, other healthy leaves, or weeds.

Impact: The Convolutional Neural Network (CNN) may accidentally pick up features from the background, like the textures of the soil, rather than focusing on the diseased leaf. This confusion can lead to misclassification, where the model detects a pattern in the background and misinterprets it as a disease lesion.

2. **Dependency on Optimal Lighting Conditions**

Computer vision models are very sensitive to pixel intensity values, which can change dramatically based on lighting.

Issue: The training dataset includes high-quality images taken in well-lit, controlled environments. However, images captured in the field often deal with various lighting challenges, such as very low light (at dusk or dawn), over-exposure (in harsh midday sunlight), or heavy shadows from other leaves.

Impact: Shadows can hide crucial disease symptoms, while over-exposure can wash away the color differences needed to tell apart similar diseases, such as distinguishing

between yellowing from nutrient deficiency and a viral infection. In these cases, the model's confidence score often drops significantly, or it makes incorrect predictions.

3. **Limited Scope of Crops and Diseases**

The system is a "closed-set" classifier, which means it can only recognize the specific classes it learned during training.

Issue: The current model is limited to a small set of crops (specifically Potato, Tomato, and Corn) and their related diseases. It cannot perform "Zero-Shot Learning" to identify plants outside this dataset.

Impact: If a user uploads a picture of a crop not in the database, like a Wheat or Rice leaf, or a different disease not included in the training set, the model will attempt to fit the image into one of the existing categories instead of rejecting it. This results in "false positives," where a completely different plant may be confidently labeled as a "Tomato Bacterial Spot."

4. **Requirement for Continuous Internet Connectivity**

The system is designed as a web application hosted on a remote server.

Issue: The image processing and inference happen on the server side, not on the user's device. Therefore, users must upload images to the cloud to receive results.

Impact: This setup creates a significant obstacle for the target audience: farmers in rural or remote areas where internet connections are often unreliable, slow, or nonexistent. Without a stable data connection, the tool becomes unusable in the field, limiting its practical use in off-grid agricultural zones.

8. CONCLUSION

The completion of the "Plant Disease Recognition using CNN" project represents a significant step in combining artificial intelligence with modern agriculture. Throughout its development, this project sought to fill a crucial gap in agriculture: the need for accessible, quick, and accurate disease diagnosis tools for farmers. We successfully developed and deployed a deep learning model that identifies plant diseases from leaf images. This achievement meets the main goals we set at the beginning.

Technical Achievement and Model Performance

The main technical success of this project is the use of the EfficientNet architecture. Unlike traditional Convolutional Neural Networks (CNNs), which often struggle to balance accuracy and computational cost, EfficientNet offered a strong solution with its compound scaling method. By using Transfer Learning, we took advantage of existing knowledge from the ImageNet dataset. This approach allowed our model to reach high validation accuracy despite having a limited number of agricultural images. The model showed great ability in distinguishing subtle disease patterns, such as the difference between Early Blight and Late Blight in potatoes. These distinctions can be hard to see with the naked eye.

Usability and Deployment

An advanced AI model is not helpful if users cannot access it easily. Integrating the model with the Streamlit framework was crucial for making the project user-friendly. The resulting web application provides a simple, "no-code" interface where users can drag and drop images to get immediate feedback. This design removes technical barriers, ensuring that farmers or agricultural extension workers with basic digital skills can use the tool effectively. We optimized the system’s response time to provide near real-time predictions, which is essential for field diagnostics.

Socio-Economic Impact and Sustainability

Beyond the code and algorithms, this project has great potential for promoting sustainable farming practices. Early and accurate detection of plant diseases enables "Precision Agriculture." Instead of spraying entire fields with broad-spectrum pesticides, which harms the environment and raises costs, farmers can focus on specific infected areas. This reduction in chemical use not only cuts production costs for farmers but also helps prevent soil degradation and groundwater contamination. Additionally, by stopping widespread crop failures through early intervention, this technology directly supports food security and economic stability for farming communities.

9. REFERENCE / BIBLIOGRAPHY

- **Keras Documentation:** https://keras.io/api/applications/efficientnet/
- **Streamlit Documentation:** https://docs.streamlit.io/
- **PlantVillage Dataset:** Kaggle.com
- Tan, M., & Le, Q. V. (2019). *EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks*.
- Mohanty, S. P., Hughes, D. P., & Salathé, M. (2016). *Using Deep Learning for Image-Based Plant Disease Detection*